

CoreYOLO: An Open Detector Built for Apple Deployment

White paper · Version 0.1.0 (alpha) · 2 September 2026

CoreYOLO is original software under the Apache License 2.0. It is inspired by the YOLO family of one-stage detectors and by the Roboflow YOLO label layout. It is **not** a fork of Ultralytics, and it is **not** affiliated with Ultralytics or Apple.

Abstract

Teams that want real-time object detection on iPhone and Mac often hit two constraints at once: they need a graph that maps cleanly onto Core ML and the Apple Neural Engine, and they need a license that allows a closed App Store binary. Widely used YOLO toolkits solve the first problem as one export among many, and they typically ship under AGPL-3.0, which is a poor fit for proprietary mobile apps.

CoreYOLO is a small, original PyTorch trainer whose product path is Core ML. It implements two detector families (a YOLOv8-style C2f + DFL graph, and a YOLO26-style C3k2 + C2PSA NMS-free graph), optional instance segmentation, Roboflow YOLO labels, a Python SDK, and a native checkpoint format. Application code loads CoreYOLO files (`.coreyolo`, trained `.pt`, or `.mlpackage`)—not Ultralytics `yolov8n.pt`. This paper describes the architecture, the train-to-device workflow, measured latency on Apple silicon, and when CoreYOLO is the right choice versus a general YOLO toolkit.

Keywords: object detection, instance segmentation, Core ML, Apple Neural Engine, Apache-2.0, YOLO, iOS

1. Introduction

YOLO-style detectors remain the default for on-device bounding boxes: one forward pass, a feature pyramid, and decoded boxes at several strides. The research lineage is public. The *product* problem is narrower. An iOS camera app does not need a 50-task training zoo. It needs (1) a static, fused convolution graph, (2) an activation the Neural Engine actually maps, (3) a host or in-graph NMS story that Vision.framework can consume, and (4) a license the legal team will sign.

Ultralytics YOLO is a strong general toolkit. It is also AGPL-3.0. Shipping those weights or that runtime inside a closed iPhone app usually requires an Ultralytics Enterprise license. Core ML export exists there, but the trainer is SiLU-first and Core ML is one backend among ONNX, TensorRT, OpenVINO, and TFLite.

CoreYOLO inverts that priority. The default activation is ReLU so fused Conv–BN–ReLU maps onto the Neural Engine. Export is an FP16 ML Program with a fixed square `imgsz`. Compute units are first-class (`gpu`, `ane`, `all`, `cpu`). The Python package and the CLI are the same internals. The iPhone artifact is a `.mlpackage` dropped into Xcode—not a Python process on the phone.

This white paper is five short chapters: problem and thesis (this page), network families, software and data, Apple deployment, and licensing plus conclusions.

2. Detector families

CoreYOLO is an **anchor-free, one-stage** network. An RGB letterboxed tensor of shape $(1, 3, \text{imgsz}, \text{imgsz})$ goes through a CSP backbone, a PAN-FPN neck, and a decoupled head at strides 8, 16, and 32 (P3 / P4 / P5). At $\text{imgsz} = 640$ that is $80^2 + 40^2 + 20^2 = 8400$ prediction cells.

Compound scaling follows the usual $n / s / m / l / x$ ladder. Nano (n) is the phone recipe: family-8 depth 0.33 and width 0.25; family-26 uses depth 0.50 on n/s/m, matching the YOLO26 compound rule. Typical use: n for real-time on phone, s on iPad/Mac, m+ when accuracy dominates.

2.1 Family 8 (default)

Family 8 is the YOLOv8 detect graph: C2f bottlenecks, SPPF, PAN-FPN, decoupled box and class branches, Distribution Focal Loss with $\text{reg_max} = 16$. Boxes are decoded from a 16-bin distribution per side, converted to xywh in pixels, concatenated with sigmoid class scores, and filtered with **class-aware NMS on the host**. That layout is what Core ML exports as $(1, 4+nc, N)$.

Because the tensors match YOLOv8n, a **one-time** remap can copy COCO-pretrained YOLOv8n (or YOLOv8n-seg) weights onto CoreYOLO-n (355/355 detect tensors; proto/cv4 for seg). The output of that remap is a CoreYOLO checkpoint (format: `coreyolo, stem.* names, SiLU`). Application code never loads `yolov8n.pt`. YOLO11n and YOLO26n cannot be copied: different blocks, different heads, and an AGPL checkpoint.

2.2 Family 26

Family 26 is an **original** C3k2 + residual SPPF + C2PSA graph with a dual head: one-to-many (TAL top-k) and one-to-one (TAL top-1) in train, DFL removed ($\text{reg_max} = 1$), and **NMS-free top-300** at inference. Eval layout is $(B, 300, 6)$ as `xyxy, conf, cls`. Official `yolo26n.pt` is not used. You train family 26 from scratch (`--family 26` or recipe `coco-n-26`).

2.3 Segmentation and training

`--task segment` adds a proto mask branch on P3 (32 prototypes, width-scaled proto channels) and per-anchor coefficients. Training adds cropped mask BCE on top of TAL + CIoU + DFL + classification BCE. Labels are Roboflow YOLO-Seg polygons. Detect-only five-value lines become filled rectangles. Mosaic is off so polygons stay valid. Core ML seg export is three outputs (`detections, mask_coeff, proto`); NMS and mask assembly stay on the host.

Assignment is Task-Aligned: $\text{score}^\alpha \times \text{IoU}^\beta$, top-k positives ($k = 10$ in the CoreYOLO loss). Optimizer is SGD with Nesterov momentum, cosine LR, 3-epoch warmup, mosaic until the last 10 epochs on detect.

Scale	Family 8 depth / width	Typical use
n	0.33 / 0.25	Phones, real-time camera
s	0.33 / 0.50	Mac / iPad
m	0.67 / 0.75	Accuracy-biased
l / x	1.00 / 1.00–1.25	Offline / max accuracy

3. Software, data, and the SDK

The package `coreyolo` (Python ≥ 3.10) is both the library and the CLI (`coreyolo train|val|predict|export|convert|coco|dummy-data`). There is no Ultralytics import on the inference path.

3.1 Labels

The dataset is the Roboflow YOLO export: `data.yaml` plus `train|valid/{images,labels}`. Each label line is normalized `class xc yc w h`, or a polygon for segmentation. The loader accepts list or dict names, `val` or `valid`, and leftover Colab paths. Official COCO 2017 can be downloaded and rewritten into that layout (`coreyolo coco`), 80 contiguous classes, crowd boxes dropped.

3.2 Native checkpoints

CoreYOLO does not use the Ultralytics pickle (`model.22.cv2...`). A native file is a dict tagged `format: "coreyolo"` with a state dict under `stem.*`, `stage*`, `head.*`, plus `nc`, `names`, `scale`, `act`, `family`, `task`. The suffix may be `.coreyolo` or `.pt`. `YOLO()` and `load_checkpoint()` reject Ultralytics `yolov8n.pt` so application code cannot accidentally depend on AGPL weights.

Optional bootstrap, once, not in the app:

```
coreyolo convert --weights yolov8n.pt --out weights/coreyolo-n-coco.coreyolo
```

3.3 Python SDK

```
from coreyolo import YOLO

model = YOLO("n")
model.train(data="data.yaml", epochs=100, imgsz=640)
model.save("weights/app.coreyolo")
results = model.predict("photo.jpg", classes="person")
model.export(imgsz=640)    # Core ML for iPhone / Mac
```

`YOLO("n")` builds an untrained nano graph. `YOLO("weights/app.coreyolo")` or a `.mlpackage` loads a trained artifact. `predict(0)` is a Mac webcam helper; it is not the iPhone runtime.

3.4 Maturity

Version 0.1.0 is alpha. Detect and instance segmentation are implemented. Pose, OBB, and classify are not. Training recipes are simpler than a large commercial zoo (no mixup/copy-paste by default). Converted YOLOv8n is the practical COCO starting point for family 8; family 26 is trained by the user. The public interface is from `coreyolo import YOLO` and the `coreyolo` CLI; they share the same internals.

4. Core ML, iPhone, and latency

Training happens on a Mac (PyTorch MPS when it works, else CUDA, else CPU). Deployment on iPhone is **Core ML**, not PyTorch. You do not need Ultralytics convert to run on a phone. Train CoreYOLO, export a `.mlpackage`, drop it into Xcode.

4.1 Export

`export` traces a fused graph (Conv–BN merged), caches grid anchors so the Core ML graph has no meshgrid/arrange, and writes an ML Program for macOS 13 / iOS 16+. Default precision is FP16. INT8 weight quant is optional. Image input is RGB `imgsz×imgsz` with 1/255 scale.

Family 8 output: decoded xywh plus class scores; **NMS on device** (Swift or Python host). Family 26 output: top-300 xyxy, conf, cls; no NMS. Segmentation: three tensors; assemble masks after NMS. The camera frame must be letterboxed to `imgsz`—the same pad/ratio the trainer used—then boxes mapped back to the original pixel frame.

```
let config = MLModelConfiguration()
config.computeUnits = .cpuAndGPU // or .cpuAndNeuralEngine / .all
let model = try VNCoreMLModel(for: CoreYOLO(configuration: config).model)
```

ReLU checkpoints map better on the Neural Engine. Converted YOLOv8 weights are SiLU; prefer GPU for those, or train ReLU from scratch for ANE.

4.2 Performance is not a single FPS number

The `.mlpackage` is the same graph on Mac and iPhone. The chip, thermals, and camera loop are not. Burst Core ML times on an M4 Pro are not iPhone camera FPS.

Local burst medians (Core ML, isolated process, CoreYOLO-n FP16 vs official YOLO26n INT8, 1 September 2026):

Config	CoreYOLO-n	YOLO26n
ANE 640×640	2.03 ms (~493 FPS burst)	1.84 ms (~545 FPS burst)
GPU-only 640×640	3.30 ms (~303 FPS)	crash (MLIR pass manager)
PyTorch MPS 640	~8 ms fused	—

A live camera app stays near display rate (often about 30 FPS) once letterbox, NMS, tracking, and UI are included. iPhone ANE/GPU is smaller than M4 Pro and will throttle. Phone recipe: scale n, start at 640, drop to 320 if the pipeline stutters, ReLU + FP16, measure with Instruments on device—not Mac burst tables. Family 8 GPU Core ML ran on this Mac; official YOLO26 GPU-only Core ML aborted. ANE remains the preferred phone compute unit when the graph is ReLU-fused.

5. Licensing, comparison, and conclusions

5.1 Why Apache-2.0 matters

CoreYOLO's train, export, and inference code is Apache-2.0. You may embed it in a proprietary iOS or macOS binary, subject to Apache notice terms. Ultralytics YOLO is AGPL-3.0: a closed App Store app that includes that code or those weights typically needs a commercial license from Ultralytics.

CoreYOLO is not a license-wash of Ultralytics. It is a separate implementation. Optional convert copies tensors from a YOLOv8n file you already have into CoreYOLO names; the file you ship is CoreYOLO. YOLO26 and YOLO11 weights are not copied.

5.2 Comparison

	CoreYOLO	Ultralytics YOLO
License	Apache-2.0	AGPL-3.0
Deploy priority	Core ML / ANE / iPhone	Many backends
Default activation	ReLU (ANE)	SiLU
Graphs	Family 8 (v8-like), family 26 (trained here)	Official v8 / 11 / 26 weights
Tasks (v0.1)	Detect, instance seg	Detect, seg, pose, OBB, cls, ...
App checkpoint	.coreyolo / CoreYOLO .pt / .mlpackage	yolov8n.pt and variants
Maturity	Alpha 0.1.0	Large ecosystem

Ultralytics is the better general trainer: more pretrained models, more tasks, more export formats, more community. CoreYOLO is the better Apple product path when AGPL is unacceptable and the ship target is Core ML.

5.3 Recommended path to iPhone

Label in Roboflow and export YOLO (or YOLO-Seg). Train with `YOLO("n").train(data="data.yaml")` or a CoreYOLO COCO recipe—not `yolov8n.pt` in the app. Save `weights/app.coreyolo`, then `model.export(imgsz=640)`. Add the `.mlpackage` to Xcode; letterbox; NMS on host for family 8; set `computeUnits` to GPU or Neural Engine. Measure on a physical iPhone.

5.4 Conclusion

CoreYOLO exists so a YOLO-style detector can be trained in the open, stored in a native checkpoint, and shipped as Core ML without taking an AGPL runtime into a closed Apple app. It does not try to replace every YOLO toolkit. It tries to be the small stack you actually put on an iPhone: C2f or C3k2, ReLU, fused convolutions, host or NMS-free decode, Apache-2.0. Further work includes richer training augs, pose/OBB if needed, tighter ANE INT8, and on-device NMS options. The public interface today is `from coreyolo import YOLO` and `coreyolo export`.